

Legal Research System - Frontend Documentation

Table of Contents

1. [Project Overview](#project-overview)
2. [Technology Stack](#technology-stack)
3. [Installation & Setup](#installation--setup)
4. [Project Structure](#project-structure)
5. [Core Components](#core-components)
6. [State Management](#state-management)
7. [API Integration](#api-integration)
8. [Styling & Animations](#styling--animations)
9. [Features](#features)
10. [Performance Optimization](#performance-optimization)
11. [Deployment](#deployment)

Project Overview

The Legal Research System is a web-based platform that enables advocates and legal professionals to quickly search, retrieve, and analyze previous court judgments and legal

precedents. The frontend provides an intuitive interface for case research, document analysis, and judgment retrieval.

Key Objectives

- Enable advocates to search for relevant legal precedents efficiently
- Display judgment details with formatted content
- Provide advanced filtering and sorting capabilities
- Support multi-document comparison
- Offer real-time search suggestions
- Display similarity analysis between cases

Technology Stack

Core Framework

- **React 18+**: Component-based UI framework
- **React Router v6**: Client-side routing
- **Vite**: Fast build tool and dev server

Styling & Animation

- **Tailwind CSS 3.x**: Utility-first CSS framework
- **Framer Motion 10.x**: Smooth animations and transitions

- **Heroicons**: Icon library for UI elements

State Management & Data Fetching

- **TanStack Query (React Query)**: Server state management
- **Zustand/Context API**: Client state management
- **Axios**: HTTP client for API calls

UI Components

- **Headless UI**: Unstyled, accessible components
- **Custom Component Library**: Built with Tailwind CSS

Development Tools

- **ESLint**: Code quality
- **Prettier**: Code formatting
- **Tailwind CSS IntelliSense**: Development enhancement

Installation & Setup

Prerequisites

- Node.js 16+ or higher

- npm or yarn package manager
- Backend API running on `http://localhost:5000`

Step 1: Clone Repository

```
git clone <repository-url> cd legal-research-frontend
```

Step 2: Install Dependencies

```
npm install # or yarn install
```

Step 3: Environment Variables

Create a `.env.local` file in the root directory:

```
VITE_API_BASE_URL=http://localhost:5000/api VITE_API_TIMEOUT=30000  
VITE_APP_NAME=Legal Research System VITE_MAX_FILE_UPLOAD_SIZE=52428800
```

Step 4: Run Development Server

```
npm run dev # or yarn dev
```

The application will be available at `http://localhost:5173`

Step 5: Build for Production

```
npm run build # or yarn build
```

Project Structure

```

legal-research-frontend/ ■■■ public/ ■■■ favicon.ico ■■■ index.html
■■■ src/ ■■■ components/ ■■■ common/ ■■■ Header.jsx ■■■
■■■ Sidebar.jsx ■■■ Footer.jsx ■■■ Loader.jsx ■■■
Toast.jsx ■■■ search/ ■■■ SearchBar.jsx ■■■
SearchFilters.jsx ■■■ SearchResults.jsx ■■■ ResultCard.jsx ■■■
■■■ judgment/ ■■■ JudgmentDetail.jsx ■■■
JudgmentMetadata.jsx ■■■ JudgmentContent.jsx ■■■
SimilarJudgments.jsx ■■■ comparison/ ■■■ ComparisonView.jsx ■■■
■■■ ComparisonPanel.jsx ■■■ DifferenceHighlight.jsx ■■■
upload/ ■■■ DocumentUpload.jsx ■■■ UploadProgress.jsx ■■■
pages/ ■■■ HomePage.jsx ■■■ SearchPage.jsx ■■■
JudgmentDetailPage.jsx ■■■ ComparisonPage.jsx ■■■ UploadPage.jsx ■■■
■■■ AboutPage.jsx ■■■ NotFoundPage.jsx ■■■ hooks/ ■■■
useSearch.js ■■■ useJudgment.js ■■■ useFetch.js ■■■
useDebounce.js ■■■ useLocalStorage.js ■■■ services/ ■■■ api.js
■■■ searchService.js ■■■ judgmentService.js ■■■
uploadService.js ■■■ analyticsService.js ■■■ store/ ■■■
searchStore.js ■■■ userStore.js ■■■ filterStore.js ■■■ utils/ ■■■
■■■ formatters.js ■■■ validators.js ■■■ constants.js ■■■
helpers.js ■■■ styles/ ■■■ global.css ■■■ tailwind.config.js ■■■
■■■ animations.css ■■■ App.jsx ■■■ main.jsx ■■■ .env.local ■■■
.eslintrc.js ■■■ tailwind.config.js ■■■ postcss.config.js ■■■
vite.config.js ■■■ package.json

```

— — —

Core Components

1. Search Bar Component

```

**File**: `src/components/search/SearchBar.jsx`

```

```
import { useState, useCallback } from 'react'; import { motion } from
'framer-motion'; import { MagnifyingGlassIcon } from
'@heroicons/react/24/solid'; import { useSearch } from '@/hooks/useSearch';
export default function SearchBar() { const [query, setQuery] = useState('');
```

```
const { performSearch } = useSearch(); const handleSearch = useCallback((e)
=> { e.preventDefault(); if (query.trim()) { performSearch(query); } },
[query, performSearch]); return ( <motion.form onSubmit={handleSearch}
initial={{ opacity: 0, y: -20 }} animate={{ opacity: 1, y: 0 }}
className="w-full max-w-2xl mx-auto" > <div className="relative"> <input
type="text" placeholder="Search judgments by case name, judge, year, or legal
issue..." value={query} onChange={(e) => setQuery(e.target.value)}
className="w-full px-4 py-3 pl-12 text-gray-900 bg-white border-2
border-gray-200 rounded-lg focus:outline-none focus:border-blue-500
transition-colors" /> <MagnifyingGlassIcon className="absolute left-3 top-3.5
w-5 h-5 text-gray-400" /> <button type="submit" className="absolute right-2
top-2 px-4 py-1.5 bg-blue-600 text-white rounded-md hover:bg-blue-700
transition-colors" > Search </button> </div> </motion.form> ); }
```

2. Search Results Component

****File**:** `src/components/search/SearchResults.jsx`

```
import { motion } from 'framer-motion'; import ResultCard from
'./ResultCard'; export default function SearchResults({ results, isLoading,
error }) { if (error) { return ( <motion.div initial={{ opacity: 0 }}
animate={{ opacity: 1 }} className="text-center py-12" > <p
className="text-red-600">Error: {error}</p> </motion.div> ); } if (isLoading)
{ return <Loader />; } if (results.length === 0) { return ( <motion.div
initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="text-center
py-12" > <p className="text-gray-500">No results found. Try different search
terms.</p> </motion.div> ); } return ( <motion.div initial={{ opacity: 0 }}
animate={{ opacity: 1 }} className="grid gap-4" > {results.map((result,
index) => ( <motion.div key={result.id} initial={{ opacity: 0, y: 20 }}
animate={{ opacity: 1, y: 0 }} transition={{ delay: index * 0.1 }} >
<ResultCard judgment={result} /> </motion.div> ))} </motion.div> ); }
```

3. Judgment Detail Component

****File**:** `src/components/judgment/JudgmentDetail.jsx`

```
import { useEffect, useState } from 'react'; import { motion } from
'framer-motion'; import { useParams } from 'react-router-dom'; import
JudgmentMetadata from './JudgmentMetadata'; import JudgmentContent from
'./JudgmentContent'; import SimilarJudgments from './SimilarJudgments';
import { useJudgment } from '@/hooks/useJudgment'; export default function
JudgmentDetail() { const { id } = useParams(); const { judgment, loading,
error, fetchJudgment } = useJudgment(); useEffect(() => { fetchJudgment(id);
```

```

}, [id]); if (loading) return <Loader />; if (error) { return ( <div
className="text-center py-12"> <p className="text-red-600">Failed to load
judgment: {error}</p> </div> ); } return ( <motion.div initial={{ opacity: 0
}} animate={{ opacity: 1 }} className="space-y-8" > <div className="grid
grid-cols-1 lg:grid-cols-3 gap-8"> <div className="lg:col-span-2">
<JudgmentContent judgment={judgment} /> </div> <div> <JudgmentMetadata
judgment={judgment} /> </div> </div> <motion.div initial={{ opacity: 0, y: 20
}} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.2 }} >
<SimilarJudgments judgmentId={id} /> </motion.div> </motion.div> ); }

```

4. Comparison View Component

****File**:** `src/components/comparison/ComparisonView.jsx`

```

import { useState } from 'react'; import { motion } from 'framer-motion';
import ComparisonPanel from './ComparisonPanel'; export default function
ComparisonView({ judgments }) { const [highlightDifferences,
setHighlightDifferences] = useState(true); return ( <motion.div initial={{
opacity: 0 }} animate={{ opacity: 1 }} className="space-y-6" > <div
className="flex justify-between items-center"> <h2 className="text-2xl
font-bold">Judgment Comparison</h2> <label className="flex items-center gap-2
cursor-pointer"> <input type="checkbox" checked={highlightDifferences}
onChange={(e) => setHighlightDifferences(e.target.checked)} className="w-4
h-4" /> <span>Highlight Differences</span> </label> </div> <div
className="grid grid-cols-1 lg:grid-cols-2 gap-6"> {judgments.map((judgment,
index) => ( <motion.div key={judgment.id} initial={{ opacity: 0, x: index ===
0 ? -20 : 20 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: index *
0.1 }} > <ComparisonPanel judgment={judgment}
highlightDifferences={highlightDifferences} /> </motion.div> ))) </div>
</motion.div> ); }

```

5. Custom Hook - useSearch

****File**:** `src/hooks/useSearch.js`

```

import { useState, useCallback } from 'react'; import { useQuery } from
'@tanstack/react-query'; import { searchService } from
'@/services/searchService'; import { useDebounce } from './useDebounce';
export function useSearch() { const [query, setQuery] = useState(''); const
[filters, setFilters] = useState({}); const debouncedQuery =
useDebounce(query, 300); const { data: results = [], isLoading, error,
refetch, } = useQuery({ queryKey: ['search', debouncedQuery, filters],
queryFn: () => searchService.search(debouncedQuery, filters), enabled:

```

```
debouncedQuery.length > 2, }); const performSearch =
useCallback((searchQuery) => { setQuery(searchQuery); }, []); const
updateFilters = useCallback((newFilters) => { setFilters(prev => ({ ...prev,
...newFilters })); }, []); return { query, results, isLoading, error,
filters, performSearch, updateFilters, refetch, }; }
```

State Management

Zustand Store Example

****File**:** `src/store/searchStore.js`

```
import { create } from 'zustand'; import { devtools, persist } from
'zustand/middleware'; export const useSearchStore = create( devtools(
persist( (set) => ({ recentSearches: [], favorites: [], addRecentSearch:
(query) => set((state) => ({ recentSearches: [query,
...state.recentSearches].slice(0, 10), })), addFavorite: (judgment) =>
set((state) => ({ favorites: [...state.favorites, judgment], })),
removeFavorite: (id) => set((state) => ({ favorites: state.favorites.filter(j
=> j.id !== id), })), clearRecentSearches: () => set({ recentSearches: [] })),
}), { name: 'search-store' } ) ) );
```

API Integration

API Service

****File**:** `src/services/api.js`

```
import axios from 'axios'; const api = axios.create({ baseUrl:
import.meta.env.VITE_API_BASE_URL, timeout: import.meta.env.VITE_API_TIMEOUT,
}); // Request interceptor api.interceptors.request.use( (config) => { const
token = localStorage.getItem('authToken'); if (token) {
config.headers.Authorization = `Bearer ${token}`; } return config; }, (error)
=> Promise.reject(error) ); // Response interceptor
api.interceptors.response.use( (response) => response.data, (error) => { if
(error.response?.status === 401) { // Handle unauthorized access
localStorage.removeItem('authToken'); window.location.href = '/login'; }
return Promise.reject(error); } }); export default api;
```

Search Service

****File**:** `src/services/searchService.js`

```
import api from './api'; export const searchService = { search: async (query,
filters = {}) => { const response = await api.get('/search', { params: { q:
query, ...filters }, }); return response.results; }, getSuggestions: async
(query) => { const response = await api.get('/search/suggestions', { params:
{ q: query }, }); return response.suggestions; }, getAdvancedSearch: async
(criteria) => { const response = await api.post('/search/advanced',
criteria); return response.results; }, };
```

Styling & Animations

Tailwind Configuration

****File**:** `tailwind.config.js`

```
export default { content: [ './index.html', './src/**/*.{js,jsx}', ], theme:
{ extend: { colors: { primary: '#1e3a8a', secondary: '#0f766e', accent:
'#dc2626', }, fontFamily: { sans: ['Inter', 'sans-serif'], serif:
['Merriweather', 'serif'], }, borderRadius: { '2xl': '1rem', }, boxShadow: {
'lg': '0 10px 15px -3px rgba(0, 0, 0, 0.1)', }, }, }, plugins: [], };
```

Framer Motion Animations

****File**:** `src/styles/animations.css`

```
/* Page transitions */ @keyframes pageEnter { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } } /* Search result hover effect */ @keyframes resultHover { 0% { transform: translateY(0); box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1); } 100% { transform: translateY(-2px); box-shadow: 0 10px 15px rgba(0, 0, 0, 0.15); } } /* Loading skeleton */ @keyframes skeleton { 0%, 100% { opacity: 1; } 50% { opacity: 0.5; } } .animate-skeleton { animation: skeleton 1.5s ease-in-out infinite; }
```

Features

1. Advanced Search

- Full-text search across judgments
- Filter by date range, judge, court, case type
- Sort results by relevance, date, or court
- Save search queries for later

2. Judgment Details

- Display complete judgment text
- Show metadata (judge, date, court, case number)
- Extract and highlight key legal principles

- Display case citations

3. Similarity Analysis

- Find similar judgments based on legal issues
- Display similarity score
- Compare key principles across judgments

4. Multi-Document Comparison

- Compare up to 3 judgments side-by-side
- Highlight differences and similarities
- Extract comparison insights

5. Document Upload

- Upload new judgments for analysis
- Automatic text extraction and indexing
- Preview before processing

6. Favorites & Collections

- Save favorite judgments
- Create custom collections
- Share collections with team members

Performance Optimization

1. Code Splitting

```
// routes/index.jsx import { lazy, Suspense } from 'react'; const SearchPage = lazy(() => import('@pages/SearchPage')); const ComparisonPage = lazy(() => import('@pages/ComparisonPage')); export function AppRoutes() { return ( <Suspense fallback={<Loader />}> <Routes> <Route path="/search" element={<SearchPage />} /> <Route path="/comparison" element={<ComparisonPage />} /> </Routes> </Suspense> ); }
```

2. Image Optimization

- Use WebP format for images
- Implement lazy loading for images
- Optimize SVG assets

3. Caching Strategy

```
// Use React Query caching const { data: results } = useQuery({ queryKey: ['search', query], queryFn: () => searchService.search(query), staleTime: 5 * 60 * 1000, // 5 minutes cacheTime: 10 * 60 * 1000, // 10 minutes });
```

4. Virtual Scrolling

For large lists, implement virtual scrolling using `react-window`:

```
import { FixedSizeList } from 'react-window'; <FixedSizeList height={600} itemCount={results.length} itemSize={100} width="100%" > {({ index, style }) => ( <div style={style}> <ResultCard judgment={results[index]} /> </div> )} </FixedSizeList>
```

Deployment

Build Configuration

```
npm run build
```

Environment-Specific Configuration

Create ``.env.production``:

```
VITE_API_BASE_URL=https://api.legalsearch.com/api VITE_API_TIMEOUT=30000
```

Deploy to Vercel

```
npm install -g vercel vercel
```

Deploy to Netlify

```
npm run build # Drag and drop the 'dist' folder to Netlify
```

Docker Deployment

****Dockerfile****:

```
FROM node:18-alpine AS builder WORKDIR /app COPY package*.json ./ RUN npm install COPY . . RUN npm run build FROM node:18-alpine WORKDIR /app RUN npm install -g serve COPY --from=builder /app/dist ./dist EXPOSE 3000 CMD ["serve", "-s", "dist", "-l", "3000"]
```

Build and run:

```
docker build -t legal-research-frontend . docker run -p 3000:3000  
legal-research-frontend
```

Best Practices

1. **Component Organization**: Keep components small and focused
2. **Prop Validation**: Use PropTypes or TypeScript for type safety
3. **Error Handling**: Always handle API errors gracefully
4. **Accessibility**: Follow WCAG guidelines for accessibility
5. **Performance**: Monitor and optimize bundle size
6. **Testing**: Implement unit and integration tests using Vitest
7. **Documentation**: Maintain updated JSDoc comments
8. **Code Review**: Ensure peer review before merging to main

Troubleshooting

Issue: API connection fails

Solution: Check API base URL in ``.env.local`` and ensure backend is running

Issue: Animations are slow

Solution: Reduce animation duration or use `will-change` CSS property

Issue: Search results not updating

Solution: Clear React Query cache: `queryClient.clear()`

Support & Resources

- **Documentation:** [React Docs](https://react.dev)
- **Tailwind CSS:** [Official Guide](https://tailwindcss.com)
- **Framer Motion:** [Animation Guide](https://www.framer.com/motion)
- **React Router:** [Routing Guide](https://reactrouter.com)

Last Updated: August 2024

Version: 1.0.0